

Computer Organization and Architecture [ENCT 303]

Chapter 2: Central Processing Unit

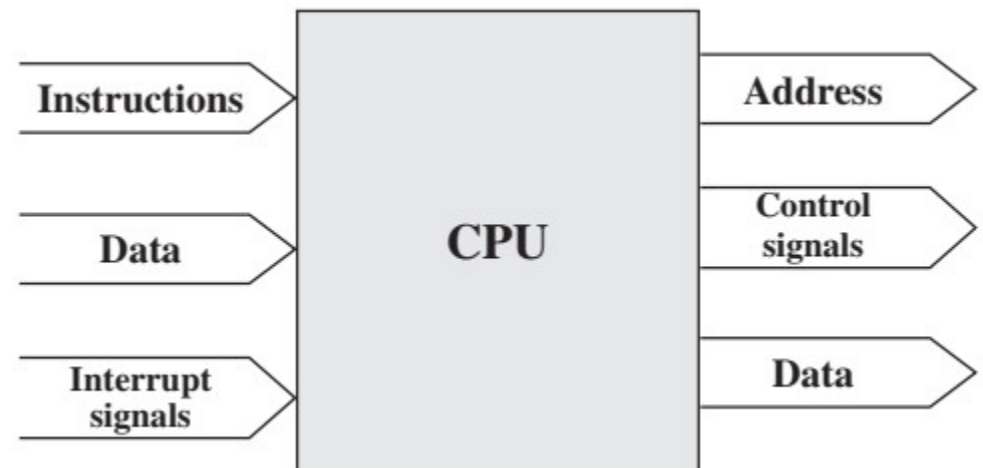
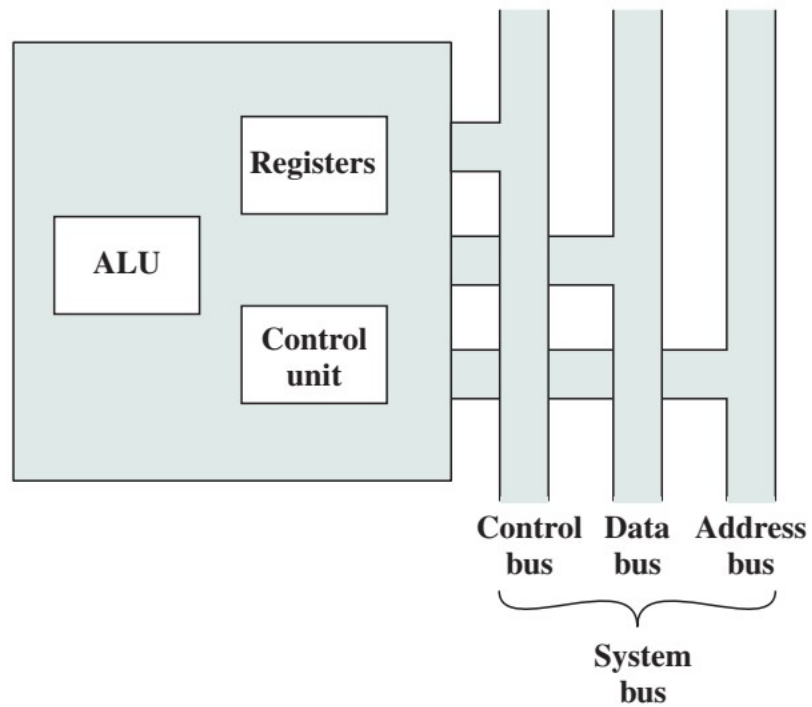
Bachelor in Computer Engineering (BCT)

Dr. Binayak Ojha
binayak.ojha@acem.edu.np

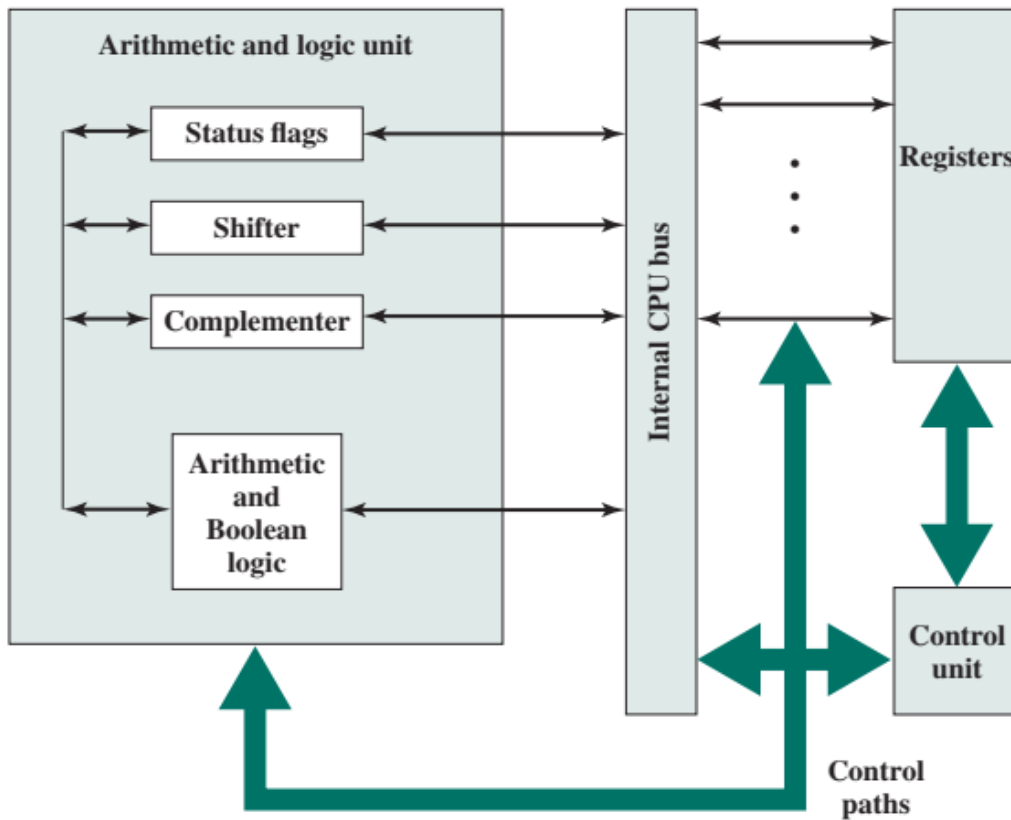
Please also refer to the class notes for complete coverage of the topic.



Processor bus organization



Processor bus organization- Internal CPU bus



Register Organization

- Registers in the processor are in the top of the memory hierarchy above main memory and cache
- Register memory → faster, smaller and more expensive per bit
- Processor registers performs two roles
 - **User-visible registers**
 - Accessable to machine or assembly
 - Used to reduce main memory references
 - **Control and status registers**
 - Used internally by control unit to control operation of processors
 - Used by privileged OS programs to control execution of programs

User-visible registers: Classification

- **General-purpose registers**
 - Can be assigned to variety of functions by the programmer
 - Can contain the operand for any opcode
- **Data registers**
 - Used only to store data, not address
- **Address registers**
 - Hold addresses
 - Examples:
 - ⇒ **Segment pointers**- hold base address of memory segment in segmented architectures
 - ⇒ **Index registers**- used in indexed addressing
 - ⇒ **Stack pointer**- a dedicated register that points to top of a stack
- **Condition codes**- referred as flag
 - Contains bits set as a result of arithmetic or logical operations
 - Example: positive, negative, zero, or overflow

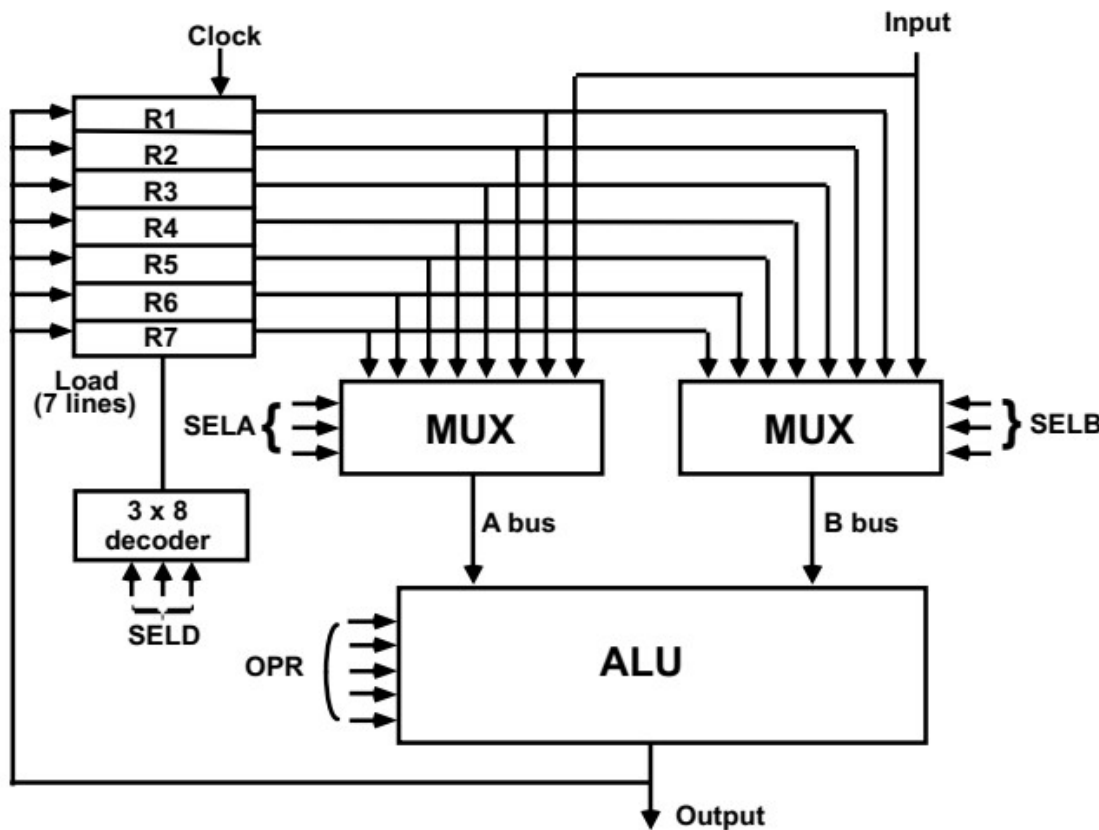
Control and Status Registers

- Processor registers used to control the operation of the processor
- Four registers are essential to instruction execution
 - **Program Counter (PC)**: Contains the address of next instruction to be fetched
 - **Instruction Register (IR)**: Contains the instruction most recently fetched.
 - **Memory Address Register (MAR)**: Contains the address of a current location in memory, usually connected directly to address bus
 - **Memory Buffer Register (MBR)**: Contains a word of data to be written to memory or data just fetched, usually connected directly to data bus

Control and Status Registers...cont.

- **Program status word (PSW)**
 - Contains status information
 - **Sign**: Contains the sign bit of the result of the last arithmetic operation
 - **Zero**: Set when the result is 0
 - **Carry**: Set if an operation resulted in a carry (addition) into or borrow(subtraction) out of a high-order bit.
 - **Equal**: Set if a logical compare result is equality
 - **Overflow**: Used to indicate arithmetic overflow
 - **Interrupt Enable/Disable**: Used to enable or disable interrupts
 - **Supervisor**: Indicates whether the processor is executing in supervisor or user mode.

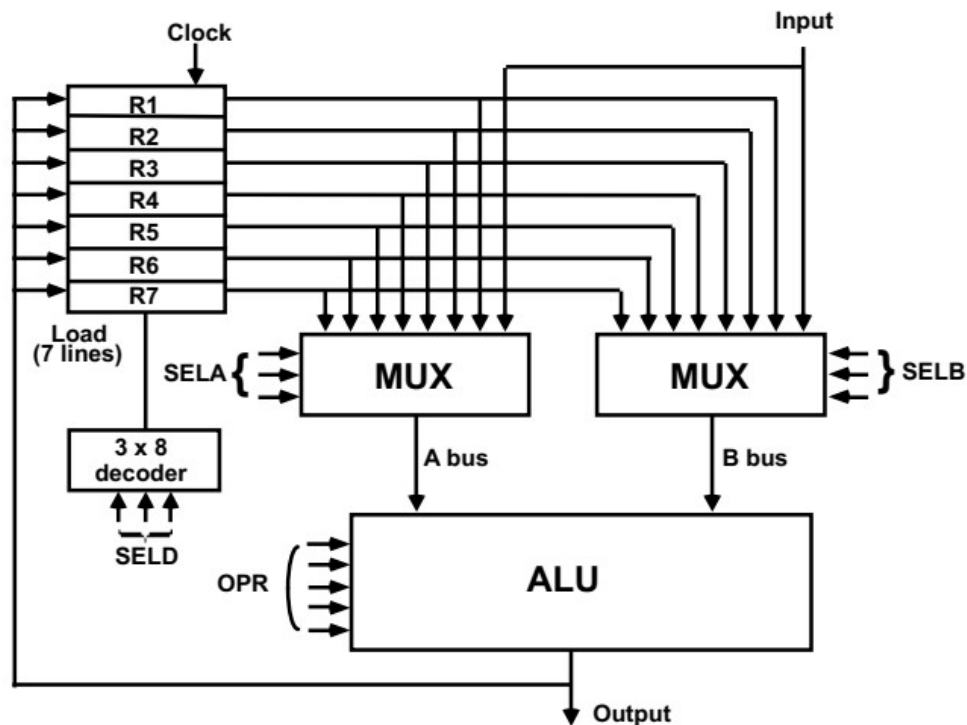
General Register Organization



- As an example, bus organization for seven CPU registers
- Output of each registers is connected to two MUX to form two buses A and B
- Selection line is each MUX select on register or input data for the particular bus
- A and B buses forms input to ALU

- Operation selected in ALU determines the arithmetic or logic microoperation to be performed
- The result of microoperation goes into the input of all registers.
- Decoder activates destination register

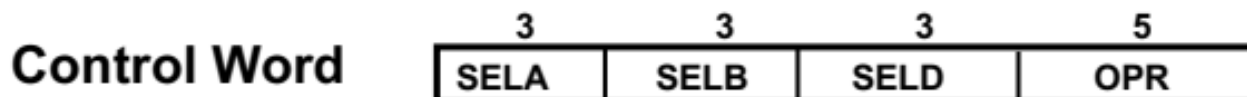
Control word and examples of microoperations



- The CU directs the information flow through ALU by
 - Selecting various components in the system
 - Selecting the function of ALU

Example : $R1 \leftarrow R2 + R3$

1. MUX A selector (SELA): to place the content of R2 into bus A
2. MUX B selector (SELB): to place the content of R3 into bus B
3. ALU operation selector (OPR): to provide the arithmetic addition $R2+R3$
4. Decoder destination selector (SELD): to transfer the content of the output bus into R1



Control word ...

- There are 14 binary selection inputs in the unit, and their combined value specifies the **control word**.
- Consist of four fields
- Three fields contain three bits each (SELA, SELB, SELD)
- One field contains five bits (OPR)

Encoding of register selection fields

Binary Code	SELA	SELB	SELD
000	Input	Input	None
001	R1	R1	R1
010	R2	R2	R2
011	R3	R3	R3
100	R4	R4	R4
101	R5	R5	R5
110	R6	R6	R6
111	R7	R7	R7

Encoding of ALU operations

OPR Select	Operation	Symbol
00000	Transfer A	TSFA
00001	Increment A	INCA
00010	ADD A + B	ADD
00101	Subtract A - B	SUB
00110	Decrement A	DECA
01000	AND A and B	AND
01010	OR A and B	OR
01100	XOR A and B	XOR
01110	Complement A	COMA
10000	Shift right A	SHRA
11000	Shift left A	SHLA

Examples of microoperations

Microoperation	Symbolic Designation				Control Word
	SELA	SELB	SELD	OPR	
$R1 \leftarrow R2 - R3$	R2	R3	R1	SUB	010 011 001 00101
$R4 \leftarrow R4 \vee R5$	R4	R5	R4	OR	100 101 100 01010
$R6 \leftarrow R6 + 1$	R6	-	R6	INCA	110 000 110 00001
$R7 \leftarrow R1$	R1	-	R7	TSFA	001 000 111 00000
$\text{Output} \leftarrow R2$	R2	-	None	TSFA	010 000 000 00000
$\text{Output} \leftarrow \text{Input}$	Input	-	None	TSFA	000 000 000 00000
$R4 \leftarrow \text{shl } R4$	R4	-	R4	SHLA	100 000 100 11000
$R5 \leftarrow 0$	R5	R5	R5	XOR	101 101 101 01100

Register Stack Organization

Stack

- Very useful feature for nested subroutines, nestep loops control
- Efficient for arithmetic expression evaluation
- Storage whic can be accessed in LIFO
- Pointer: SP → contains a binary number whose value is equal to the address of the word that is on the top of the stack
- Only PUSH and POP operations are applicable

Register Stack Organization

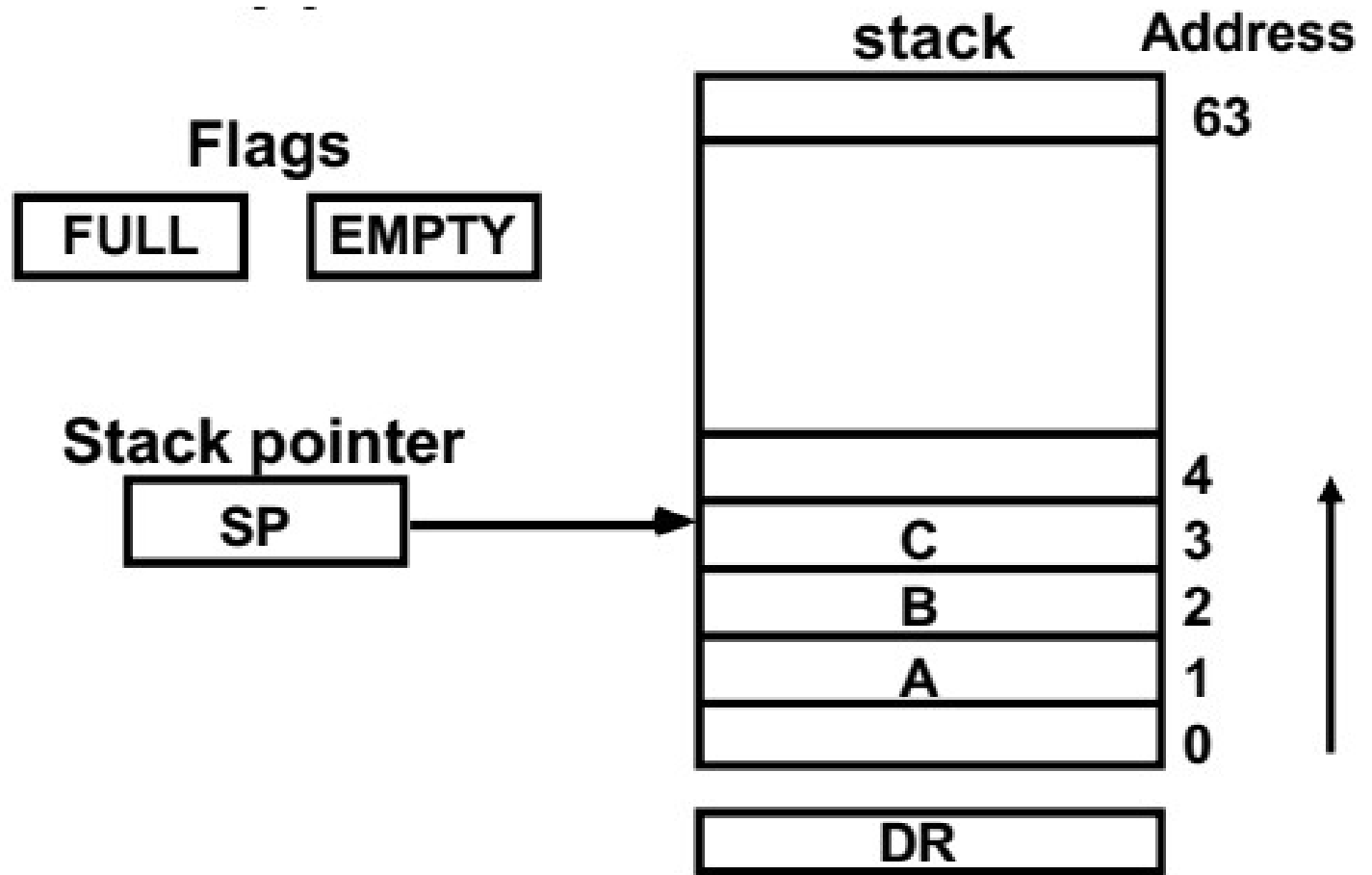


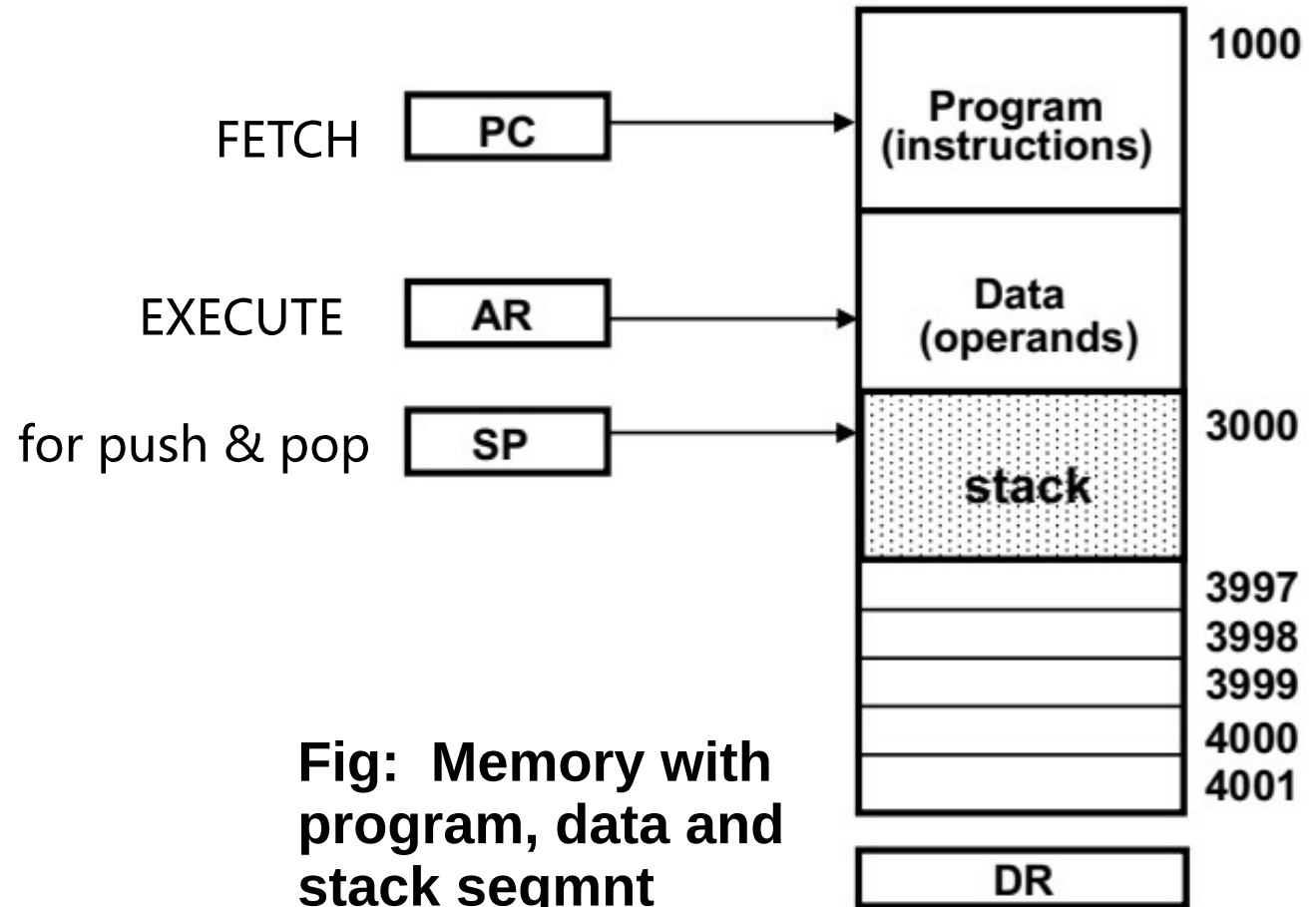
Fig: 64-word register stack

DR is the data register that holds the binary data to be written into or read out of the stack

Register Stack Organization

PUSH and POP microoperation → lecture notes

Memory Stack



A portion of memory is assigned to stack operation and a processor register is used as a stack pointer

Reverse Polish Notation

Arithmetic Expressions: A + B

A + B Infix notation

+ A B Prefix or Polish notation

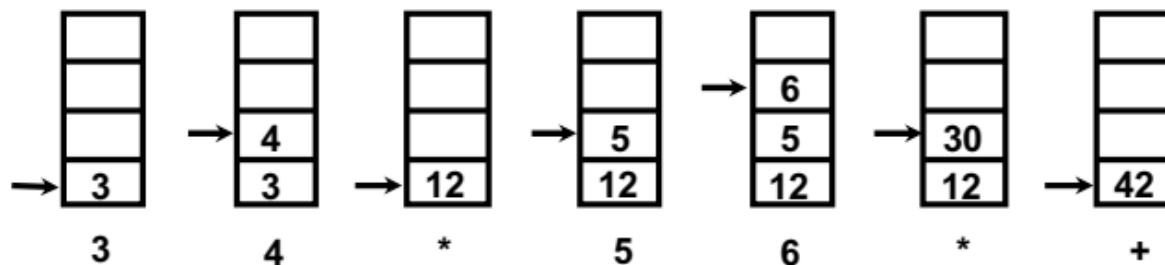
A B + Postfix or reverse Polish notation

- The reverse Polish notation is very suitable for stack manipulation**

Evaluation of Arithmetic Expressions

Any arithmetic expression can be expressed in parenthesis-free Polish notation, including reverse Polish notation

$$(3 * 4) + (5 * 6) \Rightarrow 3 \ 4 \ * \ 5 \ 6 \ * \ +$$



Addressing Modes

- Specifies a rule for interpreting or modifying the address field of the instruction before the operand is actually referenced.
- Computers use addressing mode techniques for the following purposes
 - To provide versatility to user by providing facilities as pointer to memory, counters for loop control, indexing of data and various other purposes.
 - To reduce the number of bits in the addressing field of instruction.

Effective Address (EA)

- Memory address obtained from the computation dictated by the given addressing mode
- It is the address of the operand in a computation type instruction.

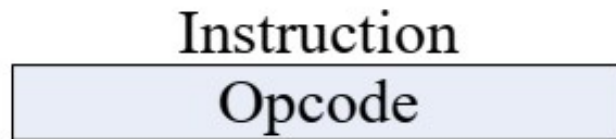
Different Addressing Modes

- i. Implied addressing mode
- ii. Immediate addressing mode
- iii. Register addressing mode
- iv. Register indirect addressing mode
- v. Auto increment and auto decrement
- vi. Direct addressing mode
- vii. Indirect addressing mode
- viii. Displacement addressing mode
- ix. Relative addressing mode
- x. Index addressing mode
- xi. Stack addressing mode

Different Addressing Modes

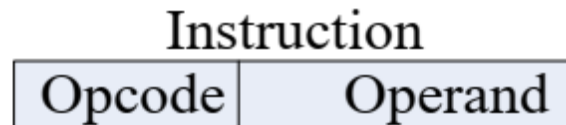
i. Implied addressing mode

- Operands are specified implicitly in definition of the instruction.
- Example: CMA i.e. complement accumulator



ii. Immediate addressing mode

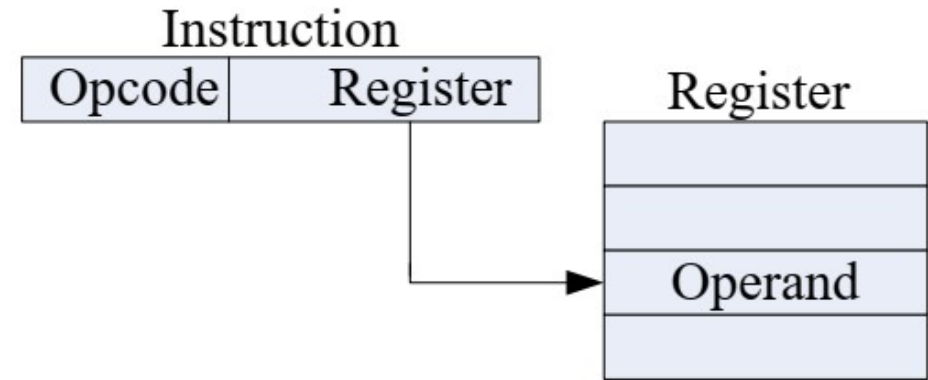
- Operand is specified in instruction itself.
- In other word, instruction has an operand rather than address
- Example: ADD 5 i.e. add 5 to the content of accumulator



Different Addressing Modes

iii. Register direct addressing mode

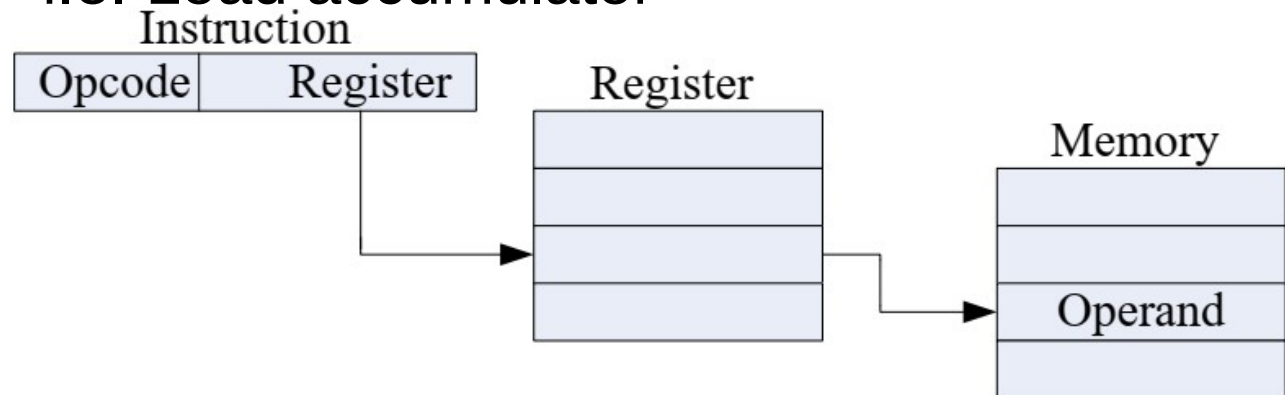
- Operands are in registers that reside within CPU
- Example: MOV A, B



Effective Address (EA) = R

iv. Register indirect addressing mode

- Instruction specifies a register in CPU whose content gives the address of operand in memory.
- Example LDAX B i.e. Load accumulator



Effective Address (EA) = (R)

Different Addressing Modes

v. Auto increment or Auto decrement addressing mode

- When address stored in the register refers to the table of data in memory, it is necessary to increment or decrement the register after every access to table.
- Example:

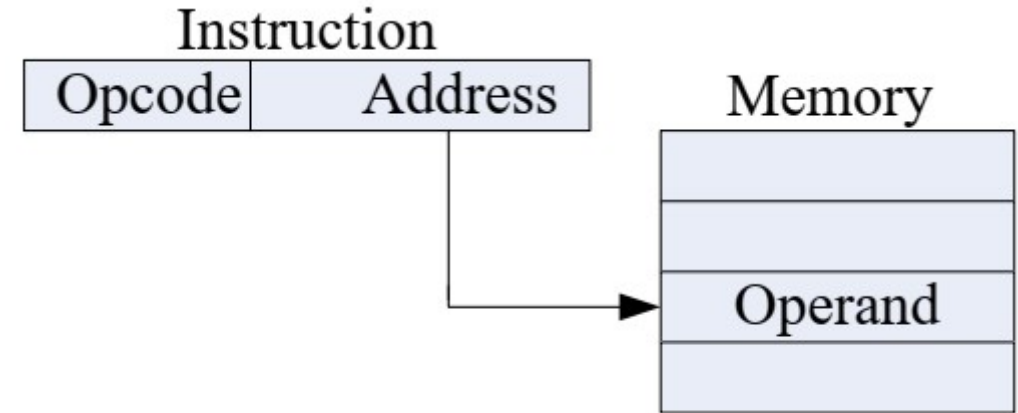
MOV R1, (R0)+ → Copy the contents of register R1 into memory at the address held in R0. Then auto increment R0

MOV R1, -(R0) → First decrement R0, then access memory at new address

Different Addressing Modes

vi. Direct addressing mode

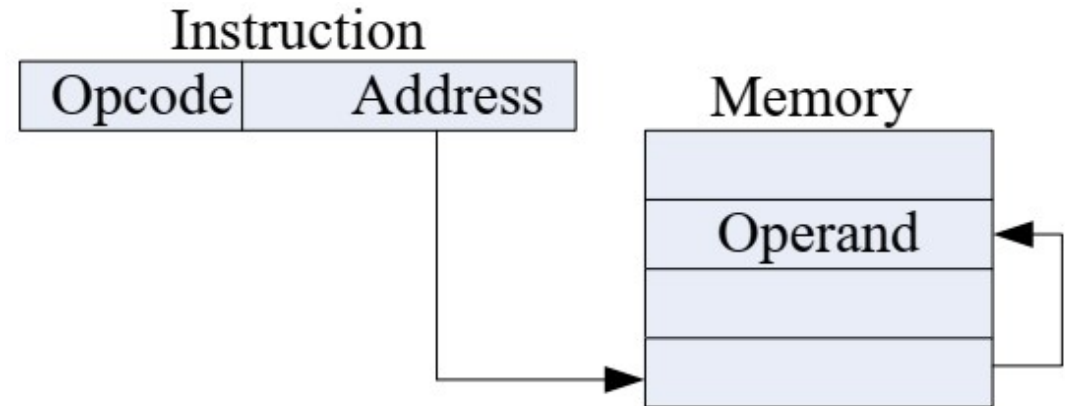
- EA is equal to address part of the instruction
- Example: LDA 4000H



Effective Address (EA) = A

vii) Indirect addressing mode

- Address field of instruction gives address where effective address is stored
- Example: ADD (A)
 - adds content of cell pointed by content of A to accumulator

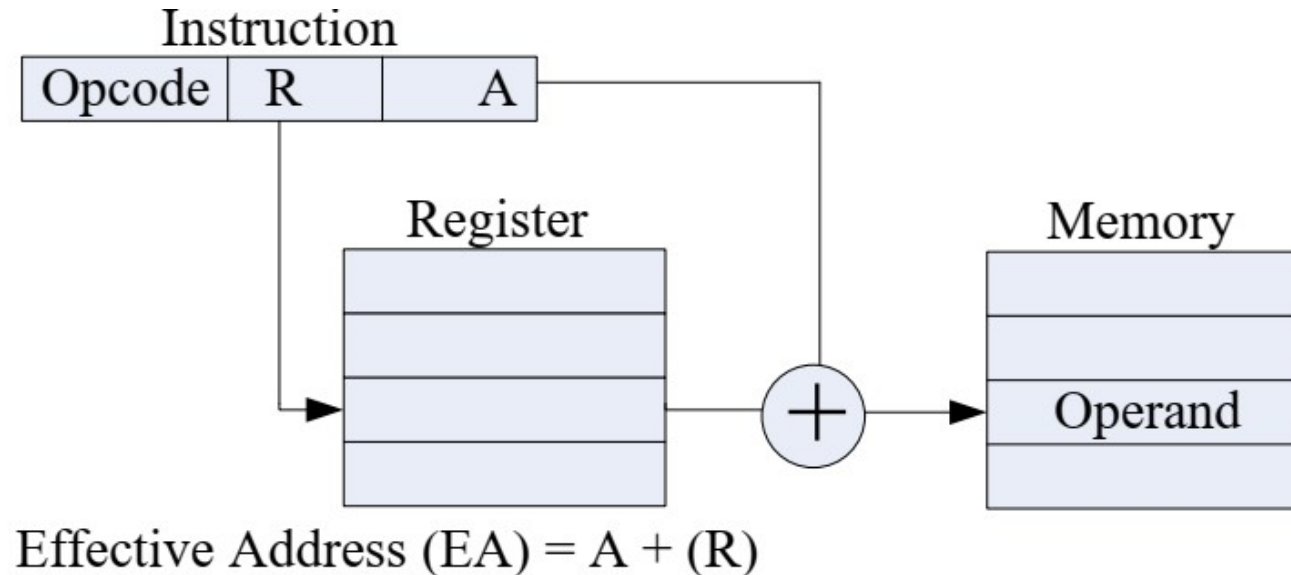


Effective Address (EA) = (A)

Different Addressing Modes

viii. Displacement addressing mode

- Address field of instruction is added to content of specific register in the CPU



Different Addressing Modes

ix. Relative addressing mode

- Content of PC is added to address part of the instruction in order to obtain EA
- Address part is usually signed (+ve or -ve)
- $EA = PC + A$

x. Indexed addressing mode

- Content of index register (XR) is added to address part of instruction to obtain EA.
- XR is special CPU register that contain index value.
- $EA = XR + A$

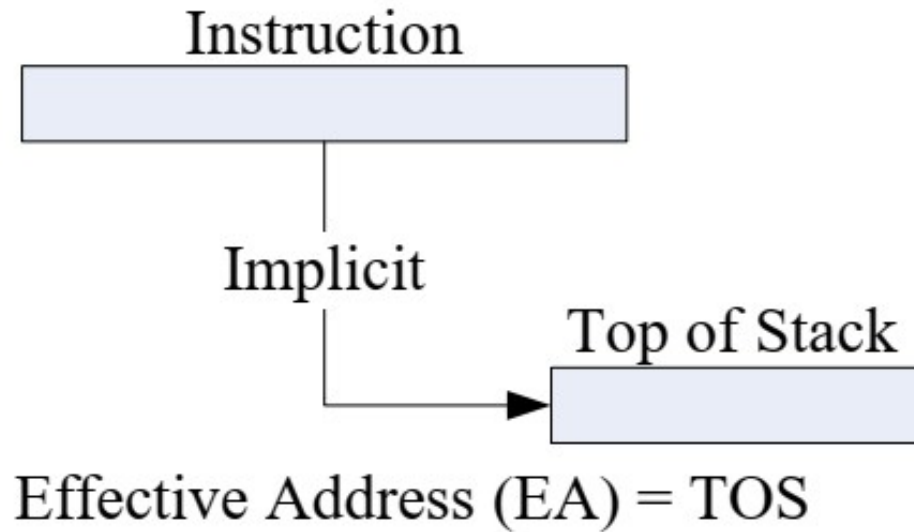
xi. Base register addressing mode

- Content of base register (BR) is added to the address part of instruction to obtain effective address.
- $EA = BR + A$

Different Addressing Modes

xii. Stack Addressing Mode

- Example: Push Ax



Numerical Example

- Two word instruction at 200 and 201 is *load to accumulator* instruction with address field equal to 500

PC = 200

R1 = 400

XR = 100

AC

Address	Memory	
200	Load to AC	Mode
201	Address = 500	
202	Next instruction	
399	450	
400	700	
500	800	
600	900	
702	325	
800	300	

Numerical Example

Addressing mode	Effective address	Content of AC
Direct address	500	800
Immediate operand	201	500
Indirect address	800	300
Relative address	702	325
Indexed address	600	900
Register	-	400
Register indirect	400	700
Autoincrement	400	700
Autodecrement	399	450

Instruction Set

- Most computer instruction can be classified into three categories

- 1) Data transfer instruction
- 2) Data manipulation instruction
- 3) Program control instruction

1. Data transfer instruction

- Moves data from one place in computer to another without changing data content
- Most common transfer are between
 - Memory and processor register
 - Processor register and input or output
 - Register themselves

Instruction Set – Data transfer instruction

- Typical data transfer instructions

Name	Mnemonic	Function
Load	LD	Memory to process register, Ac
Store	ST	Processor register to memory
Move	MOV	One register to another
Exchange	XCH	Swaps information between two registers
Input	IN	Input terminal to register
Out	OUT	Register to output terminal
Push	PUSH	Processor register to stack
Pop	POP	Stack to processor register

Instruction Set – Data manipulation instructions

- Performs operation on data and provide computational capabilities for the computer
- Performs arithmetic, logic and shift operations

Name	Mnemonic
Increment	INC
Decrement	DEC
Add	ADD
Subtract	SUB
Multiply	MUL
Divide	DIV
Add with carry	ADDC
Subtract with borrow	SUBB
Negate (2's Complement)	NEG

Instruction Set – Data manipulation instructions

- Logical Instructions

Name	Mnemonic
Clear	CLR
Complement	COM
AND	AND
OR	OR
Exclusive OR	XOR
Clear Carry	CLRC
Set Carry	SETC
Complement Carry	COMC
Enable Interrupt	EI
Disable Interrupt	DI

Instruction Set – Data manipulation instructions

- Shift Instructions

Name	Mnemonic
Logical shift right	SHR
Logical shift left	SHL
Arithmetic shift right	SHRA
Arithmetic shift left	SHLA
Rotate right	ROR
Rotate left	ROL
Rotate right through carry	RORC
Rotate left through carry	ROLC

Instruction Set – Program control instruction

Program control instructions

- Provides decision making capabilities and change the path taken by program when executed
- These instructions specify conditions for altering the content of PC
- The change in the value of PC as result of execution of PC instruction causes a break in sequence of instruction execution

Name	Mnemonic
Branch	BR
Jump	JMP
Skip	SKP
call	CALL
Return	RET
Compare (by subtraction)	CMP
Test (by ANDing)	TST

Status bit conditions

- ALU of any processor is equipped with status bits
- Also called as condition code bits or flag bits
- Fig. shows 8-bit ALU with 4 bits status register
- Four status bits are symbolised by C, S, Z and V

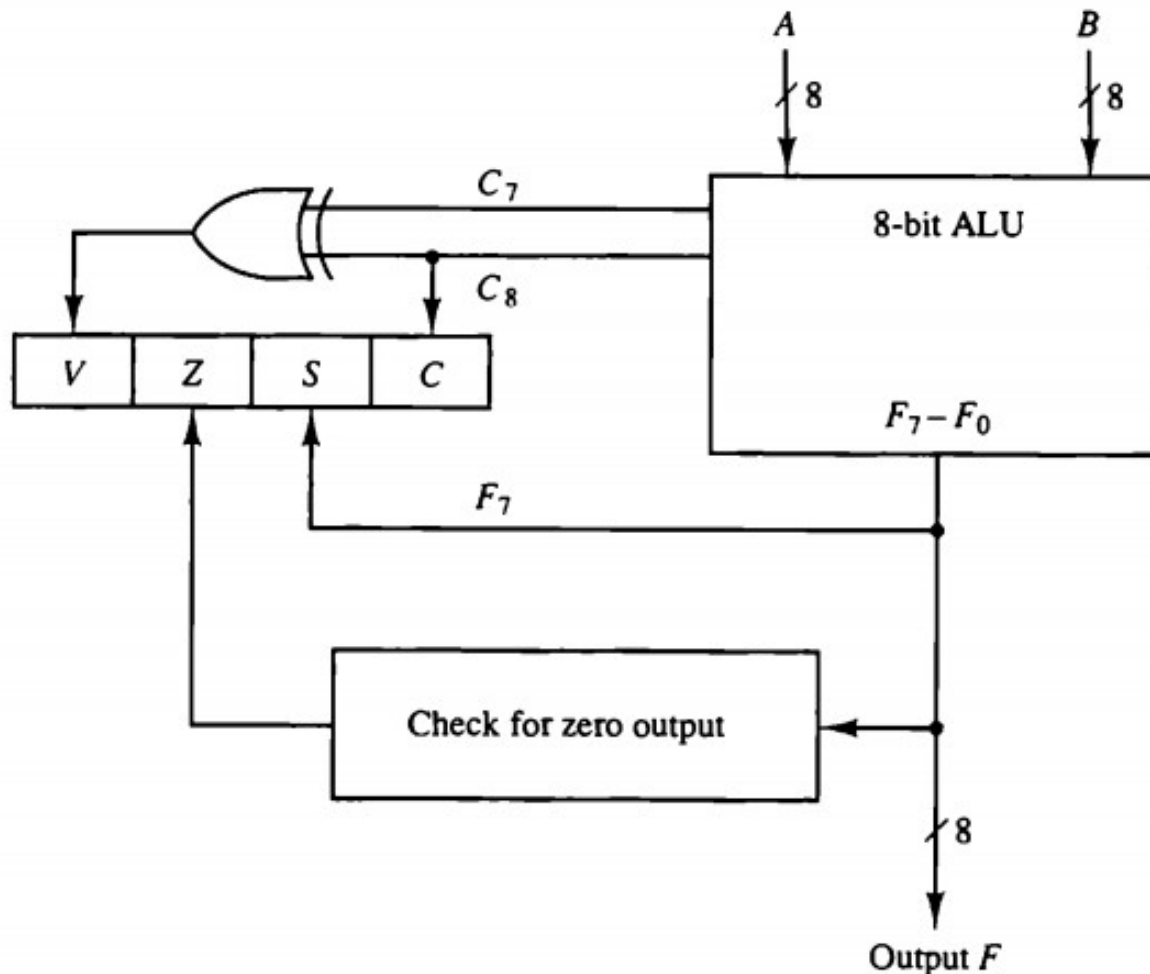


Fig: Block diagram of 8 bit ALU with 4 status bits

Status bit conditions

- 1) C (carry) is set to 1 if carry out of ALU is 1
 - 2) S (sign) is set to 1 if MSB of ALU's output is 1, 0 if the MSB is 0
 - 3) Z (zero) is set to 1, if output of ALU contains all 0
 - 4) V (overflow) is set to 1 if exclusive or of last two carries is equal to 1 and cleared to 0 otherwise, i.e. V is 1 if output is greater than +127 or less than -128.
- Status bits can be checked after ALU operation to determine certain relationship between value A and B
 - Example:
 - If $Z=1$ after $A \text{ XOR } B$, it indicates $A=B$
 - A single bit in A can be checked to determine if it is 0 or 1 by masking all bits except the bit in question and checking Z status. Let A = 0101x1100; x is unknown
 - AND operation of A with B (00010000) produces 000x0000.
 - If $x=0$, $Z=1$; if $x=1$, $Z=0$

Control Branch Instruction

Mnemonic	Branch Condition	Tested Condition
BZ	Branch if zero	Z=1
BNZ	Branch if not zero	Z=0
BC	Branch if carry	C=1
BNC	Branch if no carry	C=0
BP	Branch if plus	S=0
BM	Branch if minus	S=1
BV	Branch if overflow	V=1
BNV	Branch if no overflow	V=0
Unsigned compare condntions (A-B)		
BHI	Branch if higher	A>B
BHE	Branch if higher or equal	A≥B
BLO	Branch if lower	A<B
BLOE	Branch if lower or equal	A≤ B
BE	Branch if equal	A=B
BNE	Branch if not equal	A≠B

Control Branch Instruction

Signed compare condntions (A-B)		
BGT	Branch if greater than	$A > B$
BGE	Branch if greater or equal	$A \geq B$
BLT	Branch if less than	$A < B$
BLE	Branch if less or equal to	$A \leq B$
BE	Branch if equal	$A = B$
BNE	Branch if not equal	$A \neq B$

Subroutine call and return

- Subroutine is a self-contained sequence of instructions that performs a given computational task
- During the execution of program, a subroutine may be called to perform its function many times at various points in the main program
- Each time a subroutine is called, branch is executed to the beginning of the subroutine to start executing its set of instructions.
- After the subrouting has been executed, a branch is made back to the main program
- Subroutine call instruction consist of an operation code together with an address that specifies the beginning of subroutine.
- The instruction is executed by performing two tasks
 - The address of the next instruction available in PC is stored in temporary location (stack), so subroutine knows where to return
 - Control is transferred to beginning of subroutine

Subroutine call and return

- The last instruction of every subroutine, commonly called **return** from subroutine causes stack to pop and content of top of stack are transferred to PC.
- This way return is always to program that last called subroutines.
- Subroutine call microoperation:

$SP \leftarrow SP - 1$

Decrement Stack Pointer

$M[SP] \leftarrow PC$

Push content of PC to stack

$PC \leftarrow EA$

Transfer control to subroutine

- Subroutine return microoperation

$PC \leftarrow M[SP]$

Pop stack and transfer to PC

$SP \leftarrow SP + 1$

Increment stack pointer

Program Interrupt

- Program interrupt transfers control from a running program to and interrupt service routine (ISR) due to an internal or external request, then returns control to the original program after servicing interrupt
- Interrupt are similar to subroutine calls but differ in three ways
 - i. Interrupts are initiated by hardware/software signals, not by normal instructions (except software interrupts)
 - ii. ISR address is determined by hardware
 - iii. Interrupts save the entire CPU state, not just the program counter (PC)

Program Interrupt

- To resume correctly, CPU must restore its exact previous stat, which includes
 - Program Counter (PC)
 - All processor registers
 - Status conditions
- The collection of CPU status bits is called Program Status Word (PSW), and contains
 - All status flags
 - Interrupt enable/disable information
 - CPU mode (User or supervisor mode)

Program Interrupt

- Interrupt handling procedure:
 - CPU finishes the current instruction
 - Checks for pending interrupts
 - Pushes PC and PSW to stack
 - Loads the ISR address into the PC
 - Loads new PSW
 - Executes the interrupt service routine
- **Interrupt vectors** are specific memory address that acts as pointers, directing CPU to memory location of corresponding ISR
- It can be a single common memory location, or separate memory location for each interrupt, or Hardware-supplied ISR address
- ISR ends with return-from-interrupt instruction, which:
 - Restores PSW and PC from stack
 - Returns CPU to its previous state
 - Allows the original program to continue normally

Types of Interrupt

1) External Interrupts

- Generated by external hardware events, independent of the running program
- Common sources are
 - I/O device requests or completion signals
 - Timers (elapsed time, timeout)
 - Power failure or hardware monitoring circuits
- Characteristics
 - Asynchronous (occurs unpredictably)
 - Not tied to specific program instruction
- Examples:
 - Timeout interrupt for program stuck in endless loops
 - Power failure interrupt saving CPU state to nonvolatile memory

Types of Interrupt

2) Internal Interrupts (Traps)

- Caused by errors or exceptional condition within the program
- Example:
 - Arithmetic overflow
 - Illegal instruction
 - Memory access violation
- Characteristics
 - Synchronous with program execution
 - Occur at the same instruction every time the program is rerun
- The interrupt service routine decides the corrective action (terminate, fix or notify)

Types of Interrupt

3) Software interrupts

- Initiated deliberately by executing a special instruction
- Acts like interrupts rather than normal subroutine calls
- Common use → supervisor (system) call
- Purposes
 - Switch CPU from user mode to supervisor mode
 - Allow access to privileged operations